StudyRAG: System Architecture & File-by-File Technical Guide

An Exhaustive Reference Manual Explaining Every Source File, Configuration, Pipeline, Algorithm, and Data Artifact in the Retrieval-Augmented Generation Study Assistant

SYSTEM METADATA & SPECIFICATIONS

Project Title: StudyRAG (Retrieval-Augmented Generation Study Partner)
Target Platform: Python 3.12+ | Streamlit 1.32+ | LangChain Core & Community
Vector Storage: Facebook AI Similarity Search (FAISS) & ChromaDB
Dense Embeddings: HuggingFace sentence-transformers/all-MiniLM-L6-v2 (384-d)
Inference Engines: Groq Cloud API, OpenAI API (GPT-4o), and Local Ollama
Document Ingestion: PyMuPDF (MuPDF v1.23+) Coordinate & Text Parsing
Publication Date: October 2026 | Comprehensive Handover Edition

Executive Purpose: This document provides an exhaustive, granular code-level breakdown of every single file in the StudyRAG repository. Designed as an authoritative developer reference, it covers each file's specific functional purpose, class/method definitions, inter-module dependencies, algorithmic thresholds, error-handling mechanisms, and execution criticality. From low-level PDF byte cleaning and recursive text splitting to high-level multi-provider LLM orchestration, anti-hallucination guardrails, asynchronous background worker threads, and modular Streamlit UI components, every component is detailed with complete implementation context.

ARCHITECTURE HIGHLIGHTS

Key Architectural Pillars:
1. **Strict Document Grounding:** No answer is generated unless relevant context chunks exceed the mathematical relevance threshold ($\tau = 0.20$).
2. **Zero UI Blocking:** Long-running LLM generation tasks (quizzes, flashcards, summaries) run in isolated background daemon threads (`threading.Thread`) managed via Streamlit script contexts.
3. **Dual Vector Backends:** Plug-and-play support for FAISS (in-memory L2 distance search with serialization) and ChromaDB (SQLite-backed persistent collection storage with HNSW Cosine space).
4. **Modular Custom UI:** Highly customized CSS token system with full Dark/Light mode switching, responsive glassmorphism containers, and interactive flashcard 3D flip dynamics.

Chapter 1: Repository Architecture & Directory Map

StudyRAG is engineered with strict separation of concerns across clean architectural layers: **Ingestion**, **Embeddings**, **Vector Storage**, **Retrieval**, **Generation**, **RAG Orchestration**, **Evaluation**, **Utilities**, **User Interface**, and **Automated Tests**. The following tree illustrates all 66+ verified files documented in this handbook:

REPOSITORY STRUCTURE HIERARCHY

rag-study-assistant/ ■■■ **app.py** *# Central Streamlit web application & background thread manager* ■■■ **requirements.txt** *# Python dependency manifest & version specifications* ■■■ **pytest.ini** *# Pytest root configuration (pythonpath = .)* ■■■ **LICENSE** *# MIT Open Source License* ■■■ **README.md** *# Public repository documentation & setup overview* ■■■ **config/** ■ ■■■ **settings.yaml** *# Application defaults (chunk size, top_k, threshold)* ■ ■■■ **ui_text.yaml** *# Externalized UI text, labels, and localized tooltips* ■■■ **src/** ■ ■■■ **ingestion/** *# Document extraction, normalization, and chunking* ■ ■ ■■■ **__init__.py**, **pdf_loader.py**, **document_cleaner.py**, **chunker.py**, **ingestion_pipeline.py** ■ ■■■ **embeddings/** *# Local HuggingFace embedding manager* ■ ■ ■■■ **__init__.py**, **embedding_manager.py** ■ ■■■ **vectorstore/** *# FAISS and Chroma vector storage engines* ■ ■ ■■■ **__init__.py**, **faiss_store.py**, **chroma_store.py**, **vector_store_manager.py** ■ ■■■ **retrieval/** *# Semantic search, score normalization & thresholding* ■ ■ ■■■ **__init__.py**, **query_processor.py**, **score_filter.py**, **retriever.py**, **retrieval_pipeline.py** ■ ■■■ **generation/** *# LLM multi-provider factory & prompt templates* ■ ■ ■■■ **__init__.py**, **llm_manager.py**, **prompt_templates.py**, **study_generator.py** ■ ■■■ **rag/** *# End-to-end RAG workflow & fallback refusal logic* ■ ■ ■■■ **__init__.py**, **rag_pipeline.py** ■ ■■■ **evaluation/** *# Information retrieval metrics (Precision@K, Recall@K, HitRate)* ■ ■ ■■■ **__init__.py**, **retrieval_metrics.py** ■ ■■■ **utils/** *# Shared utilities (logging, hashing, history persistence)* ■ ■ ■■■ **__init__.py**, **logger.py**, **file_utils.py**, **history_manager.py** ■ ■■■ **ui/** *# Modern component-based Streamlit UI system* ■ ■■■ **__init__.py**, **text_provider.py**, **styles.py** ■ ■■■ **components/** (12 modular folders: base, badge, button, card, dashboard, emptyState, ■ errorState, input, loading, notifications, tasks, welcome) ■■■ **static/css/style.css** *# Global styles, scrollbars, typography resets* ■■■ **tests/** *# Pytest automated test suite (9 test modules, 27+ assertions)* ■■■ **data/** *# Persistent state (history.json, uploaded PDFs, vector indices)*

System Dataflow Lifecycle

The end-to-end dataflow transitions through distinct phases:

1. **Upload & Hash**: User uploads a PDF; file_utils.py computes SHA-256 for deduplication.
2. **Extraction & Cleaning**: pdf_loader.py extracts text with page numbers; document_cleaner.py strips excess whitespace.
3. **Semantic Chunking**: chunker.py partitions text recursively into 500-char blocks with 50-char overlap.
4. **Embedding & Indexing**: embedding_manager.py generates 384-d vectors; vector_store_manager.py saves to FAISS/Chroma.
5. **Query Sanitization & Retrieval**: query_processor.py cleans user question; retriever.py fetches top-k nearest chunks.
6. **Score Normalization & Anti-Hallucination**: score_filter.py maps raw distance to [0.0, 1.0] and rejects chunks below τ=0.20.
7. **Prompt Synthesis & Generation**: prompt_templates.py formats context; llm_manager.py generates cited answers.
8. **Asynchronous Task Dispatch**: Study tools (quizzes, cards) run in background threads, persisting outputs to history.json.

Chapter 2: Root Files & Application Configuration

File: app.py [CRITICAL]

Role & Layer: Presentation Layer / Application Orchestration / Entry Point

Detailed Purpose & Use:
The primary runtime entry point for StudyRAG. Implements the entire Streamlit multi-page interface, manages reactive session state variables, configures page layouts and themes, handles background thread spawning (`threading.Thread` with Streamlit script execution context via `add_script_run_ctx`), coordinates document ingestion and vectorstore rebuilding, and dispatches views: Welcome, Document Manager, Chat Assistant, Quiz Generator, Flashcard Deck, Chapter Summarizer, and Study History.

Key Exports & Signatures:
```
load_config() -> dict get_embedding_manager(model_key: str) -> EmbeddingManager (cached)
get_llm_manager(provider, model_name, temp) -> LLMManager (cached) ensure_vector_store_loaded(...) ->
VectorStoreManager delete_study_material(doc_name, ...) -> None worker_generate_quiz(job_id, topic, ...) -> None
worker_generate_flashcards(job_id, topic, ...) -> None worker_generate_summary(job_id, topic, ...) -> None
main() -> None; render_*_page() handlers
```
Dependencies & Interactions: Imports all `src.*` submodules (`ingestion_pipeline`, `embedding_manager`, `vector_store_manager`, `retrieval_pipeline`, `llm_manager`, `rag_pipeline`, `study_generator`, `history_manager`, `file_utils`, `styles`, `text_provider`, `components`).

Failure Modes & Defenses: Gracefully handles missing directories by auto-creating them; catches LLM timeouts without crashing the UI; isolates background worker thread exceptions by logging them and updating job status to 'failed'.

File: config/settings.yaml [CRITICAL]

Role & Layer: Configuration Manifest / Default Hyperparameters

Detailed Purpose & Use:
Defines all default hyperparameters and application-level constants in YAML format. Contains configuration blocks for `app` (title, version), `chunking` (strategy: recursive, chunk_size: 500, chunk_overlap: 50), and `retrieval` (top_k: 5, similarity_threshold: 0.20). Read at startup by `app.py` via `load_config()` to establish baseline application settings.

Key Exports & Signatures:
```
YAML Key-Value Mapping: app.*, chunking.*, retrieval.*
```
Dependencies & Interactions: Loaded by `app.py` and `tests/test_retrieval.py` via `yaml.safe_load()`.

Failure Modes & Defenses: If missing or malformed, `app.py` falls back to hardcoded defaults (500 size, 50 overlap, 0.2 threshold).

File: config/ui_text.yaml [HIGH]

Role & Layer: Localization & Presentation Copy Manifest

Detailed Purpose & Use:
Central repository for all user-facing strings, microcopy, headers, button labels, tooltips, and instructions across every UI component and page. Completely decouples visual presentation text from backend logic, enabling zero-code text adjustments, localization, or rebranding without modifying Python source.

Key Exports & Signatures:
```
Nested YAML hierarchy: app.*, sidebar.*, welcome.*, documents.*, chat.*, quiz.*, flashcards.*, summary.*,
history.*
```
Dependencies & Interactions: Read by `src/ui/text_provider.py` via `get_ui_text(*keys)`.

Failure Modes & Defenses: If a key is missing, `get_ui_text()` returns a specified fallback default string without breaking UI flow.

File: requirements.txt [CRITICAL]

Role & Layer: Dependency Manifest / Build Specification

Detailed Purpose & Use:
Specifies exact minimum version constraints for all Python package dependencies required to run StudyRAG in development and production environments. Enforces compatibility for Streamlit, PyMuPDF, LangChain, HuggingFace embeddings, sentence-transformers, FAISS CPU, ChromaDB, Plotly, and Pytest.

Key Exports & Signatures:
`streamlit>=1.32.0, PyMuPDF>=1.23.0, langchain>=0.1.13, langchain-community>=0.0.29, langchain-huggingface>=0.0.1, langchain-openai>=0.1.0, langchain-ollama>=0.1.0, langchain-text-splitters>=0.0.1, sentence-transformers>=2.5.1, faiss-cpu>=1.8.0, chromadb>=0.4.24, numpy>=1.26.0, pandas>=2.2.1, pyyaml>=6.0.1, python-dotenv>=1.0.1, pytest>=8.0.0`
Dependencies & Interactions: Used by `pip install -r requirements.txt` during virtual environment bootstrapping.

Failure Modes & Defenses: Outdated or conflicting versions trigger runtime import errors in LangChain or FAISS.

File: pytest.ini [MEDIUM]

Role & Layer: Test Runner Configuration

Detailed Purpose & Use:
Defines global execution settings for pytest. Configures `pythonpath = .` so that all unit and integration tests in the `tests/` directory can resolve absolute imports from `src.*` without requiring PYTHONPATH shell hacks or local package installation.

Key Exports & Signatures:
`[pytest] pythonpath = .`
Dependencies & Interactions: Executed automatically whenever `pytest` is invoked in terminal.

Failure Modes & Defenses: Without this file, running `pytest tests/` fails with `ModuleNotFoundError: No module named 'src'`.

File: LICENSE [LOW]

Role & Layer: Legal Terms / Open Source Governance

Detailed Purpose & Use:
Standard MIT License granting users full rights to use, copy, modify, merge, publish, and distribute StudyRAG.

Key Exports & Signatures:
`MIT License legal text`
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: README.md [MEDIUM]

Role & Layer: Developer & User Documentation

Detailed Purpose & Use:
Primary public documentation file for the repository. Details project background, feature highlights, installation instructions, environment setup (`.env`), supported models (Groq, OpenAI, Ollama), testing procedures, and architectural diagrams.

Key Exports & Signatures:
`Markdown documentation`
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

Chapter 3: Document Ingestion Layer (src/ingestion/)

The Ingestion layer is responsible for raw document ingestion, file validation, text extraction, syntactic normalization, and recursive chunking. It transforms heterogeneous multi-page academic PDFs into discrete, metadata-tagged text segments ready for dense vector embedding.

File: src/ingestion/__init__.py [LOW]

Role & Layer: Ingestion Module Package Marker & Facade

Detailed Purpose & Use:
Marks `src.ingestion` as a Python package, enabling clean submodule imports across the codebase.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/ingestion/pdf_loader.py [CRITICAL]

Role & Layer: PDF Document Extractor & Page Parser

Detailed Purpose & Use:
Extracts raw text page-by-page from PDF files using PyMuPDF (`fitz`). Implements robust validation for file existence, non-PDF file extensions, and corrupted or password-encrypted documents. Detects empty PDFs (0 pages) and warns if a document contains no extractable text (e.g., scanned bitmap images). Associates each extracted page with 1-indexed page number and source filename metadata.

Key Exports & Signatures:
```
load_pdf(pdf_path: str | Path) -> List[Dict[str, Any]] Output item: {'text': str, 'source': str, 'page': int}
```
Dependencies & Interactions: PyMuPDF (`pymupdf` or fallback `fitz`), `pathlib.Path`, `src.utils.logger`.

Failure Modes & Defenses: Raises `FileNotFoundError` if path does not exist; raises `ValueError` if file extension is not `.pdf` or if the document cannot be opened due to encryption/corruption.

File: src/ingestion/document_cleaner.py [CRITICAL]

Role & Layer: Text Sanitizer & Blank Page Remover

Detailed Purpose & Use:
Sanitizes extracted document text before chunking. Strips redundant horizontal whitespace and tabs (`[ \t]+ -> ' '`), compresses three or more consecutive newlines into standard double newlines (`\n{3,} -> \n\n`) to preserve paragraph semantics, and trims outer whitespace. Automatically filters out blank pages or pages containing only whitespace characters.

Key Exports & Signatures:
```
clean_text(text: str) -> str clean_documents(documents: List[Dict[str, Any]]) -> List[Dict[str, Any]]
```
Dependencies & Interactions: Python standard library `re`, `typing`.

Failure Modes & Defenses: Gracefully handles empty strings or None values by returning empty strings without raising exceptions.

File: src/ingestion/chunker.py [CRITICAL]

Role & Layer: Recursive Character Semantic Text Splitter

Detailed Purpose & Use:
Splits cleaned documents into semantic chunks using LangChain's `RecursiveCharacterTextSplitter`. Applies a hierarchical separator list: paragraph breaks (`\n\n`), line breaks (`\n`), sentence ends (`. `), spaces (` `), and raw characters (`""`). Preserves all upstream source and page metadata and attaches a deterministic, monotonic integer `chunk_id` to every generated chunk.

Key Exports & Signatures:
`create_chunks(documents: List[Dict], chunk_size: int = 500, chunk_overlap: int = 50) -> List[Dict] Output item: {'text': str, 'source': str, 'page': int, 'chunk_id': int}`
Dependencies & Interactions: `langchain_text_splitters.RecursiveCharacterTextSplitter`, `typing`.

Failure Modes & Defenses: Strictly validates chunk parameters: raises `ValueError` if `chunk_size <= 0`, `chunk_overlap < 0`, or `chunk_overlap >= chunk_size`.

File: src/ingestion/ingestion_pipeline.py [CRITICAL]

Role & Layer: End-to-End Ingestion Pipeline Coordinator

Detailed Purpose & Use:
Coordinates the complete ingestion sequence for a single PDF: loads raw pages via `load_pdf()`, sanitizes text via `clean_documents()`, and partitions text via `create_chunks()`. Emits structured logging at each stage and handles empty document edge cases gracefully.

Key Exports & Signatures:
`process_pdf(pdf_path: str | Path, chunk_size: int = 500, chunk_overlap: int = 50) -> List[Dict[str, Any]]`
Dependencies & Interactions: `pdf_loader`, `document_cleaner`, `chunker`, `src.utils.logger`.

Failure Modes & Defenses: Logs any unexpected exceptions during the pipeline and re-raises them for upstream UI alert handling.

Chapter 4: Semantic Embeddings Layer (src/embeddings/)

File: src/embeddings/__init__.py [LOW]

Role & Layer: Embeddings Package Marker

Detailed Purpose & Use:
Package marker file for the embeddings layer.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/embeddings/embedding_manager.py [CRITICAL]

Role & Layer: Dense Vector Embedding Model Manager

Detailed Purpose & Use:
Initializes and manages local semantic embedding models using `langchain_huggingface.HuggingFaceEmbeddings`. Maintains a supported model registry mapping model keys to HuggingFace hub identifiers: `minilm` (`sentence-transformers/all-MiniLM-L6-v2`, 384-d) and `bge` (`BAAI/bge-small-en-v1.5`, 384-d). Implements dynamic dimension caching via `get_embedding_dimension()` by embedding a sample probe query.

Key Exports & Signatures:
```
class EmbeddingManager(model_key: str = 'minilm') .get_embeddings() -> HuggingFaceEmbeddings .get_model_name()
-> str .get_embedding_dimension() -> int
```
Dependencies & Interactions: `langchain_huggingface.HuggingFaceEmbeddings`, `src.utils.logger`.

Failure Modes & Defenses: Falls back to 'minilm' if an unrecognized model key is passed; raises `RuntimeError` if HuggingFace fails to download or load model weights into memory.

Chapter 5: Vector Storage Layer (src/vectorstore/)

File: src/vectorstore/__init__.py [LOW]

Role & Layer: Vector Store Package Marker

Detailed Purpose & Use:
Package marker file for vector storage components.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/vectorstore/vector_store_manager.py [CRITICAL]

Role & Layer: Vector Store Factory & Adapter Coordinator

Detailed Purpose & Use:
Provides a unified, provider-agnostic factory and manager interface for vector database backends. Instantiates either `FAISSStore` or `ChromaStore` based on configuration, converts raw ingestion dictionaries into LangChain `Document` objects with standardized metadata (`source`, `page`, `chunk_id`), and provides unified wrappers for adding, saving, loading, querying, and resetting indices.

Key Exports & Signatures:
```
class VectorStoreManager(provider: str, embeddings: Embeddings, persist_directory_base: str)
.raw_chunks_to_documents(chunks: List[Dict]) -> List[Document] .add_documents(documents: List[Document]) -> None
.save() -> None; .load() -> bool; .reset() -> None .similarity_search(query: str, k: int = 5) ->
List[tuple[Document, float]]
```
Dependencies & Interactions: `langchain_core.documents.Document`, `faiss_store.FAISSStore`, `chroma_store.ChromaStore`.

Failure Modes & Defenses: Raises `ValueError` if an unsupported provider string is passed (e.g. not 'faiss' or 'chroma').

File: src/vectorstore/faiss_store.py [HIGH]

Role & Layer: FAISS In-Memory & Serialized Index Wrapper

Detailed Purpose & Use:
Wraps Facebook AI Similarity Search (`langchain_community.vectorstores.FAISS`). Performs blazing-fast in-memory exact L2 Euclidean distance searches. Persists indices to disk as binary index files (`index.faiss`) and metadata pickles (`index.pkl`). Implements safe disk loading with `allow_dangerous_deserialization=True` and handles complete directory resets via `shutil.rmtree`.

Key Exports & Signatures:
```
class FAISSStore(embeddings, persist_directory='data/vectorstores/faiss/') .add_documents(documents:
List[Document]) -> None .save() -> None; .load() -> bool; .reset() -> None .similarity_search(query: str, k:
int) -> List[tuple[Document, float]]
```
Dependencies & Interactions: `langchain_community.vectorstores.FAISS`, `pathlib.Path`, `shutil`, `src.utils.logger`.

Failure Modes & Defenses: Returns False from `.load()` if files do not exist; catches and logs deserialization corruptions.

File: src/vectorstore/chroma_store.py [HIGH]

Role & Layer: ChromaDB SQLite & HNSW Persistent Store Wrapper

Detailed Purpose & Use:
Wraps ChromaDB (`langchain_community.vectorstores.Chroma`). Stores embeddings and documents in an embedded SQLite database (`chroma.sqlite3`) and an HNSW segment index directory. Configured explicitly for cosine similarity space via `collection_metadata={'hnsw:space': 'cosine'}`. Automatically persists mutations to disk and supports full collection deletion and index resetting.

Key Exports & Signatures:
```
class ChromaStore(embeddings, persist_directory='data/vectorstores/chroma/') .add_documents(documents:
List[Document]) -> None .save() -> None; .load() -> bool; .reset() -> None .similarity_search(query: str, k:
int) -> List[tuple[Document, float]]
```
Dependencies & Interactions: `langchain_community.vectorstores.Chroma`, `pathlib.Path`, `shutil`, `src.utils.logger`.

Failure Modes & Defenses: Returns False from `.load()` if `chroma.sqlite3` does not exist on disk; catches collection reset errors.

Chapter 6: Retrieval & Filtering Layer (src/retrieval/)

File: src/retrieval/__init__.py [LOW]

Role & Layer: Retrieval Package Marker

Detailed Purpose & Use:
Package marker file for retrieval components.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/retrieval/query_processor.py [CRITICAL]

Role & Layer: Query Sanitizer & Input Boundary Validator

Detailed Purpose & Use:
Sanitizes and validates natural language user input before submitting queries to vector search. Normalizes multi-line whitespace and tab characters into single spaces, trims padding, and enforces both non-empty checks and a maximum query length bound (default: 1,000 characters) to prevent prompt injection attacks or memory over-allocation.

Key Exports & Signatures:
```
process_query(query: str, max_query_length: int = 1000) -> str
```
Dependencies & Interactions: `re` regex engine.

Failure Modes & Defenses: Raises `ValueError` with clear user-facing messages if query is empty, whitespace-only, or exceeds length limit.

File: src/retrieval/score_filter.py [CRITICAL]

Role & Layer: Score Normalization & Anti-Hallucination Guardrail

Detailed Purpose & Use:
Solves the metric disparity between vector store backends by normalizing raw distance scores to a unified 0.0 to 1.0 relevance scale (1.0 = identical). For FAISS (L2 distance d), computes score = 1.0 / (1.0 + d). For Chroma (Cosine distance d), computes score = max(0.0, min(1.0, 1.0 - d)). Filters candidate chunks against the similarity threshold (τ = 0.20) and sorts descending.

Key Exports & Signatures:
```
normalize_score(raw_score: float, provider: str) -> float filter_by_threshold(results: List[tuple], provider: str, threshold: float) -> List[Dict]
```
Dependencies & Interactions: `langchain_core.documents.Document`, `math`.

Failure Modes & Defenses: Bounds all scores strictly between 0.0 and 1.0; returns empty list if no chunks meet threshold.

File: src/retrieval/retriever.py [CRITICAL]

Role & Layer: Source-Filtered Vector Search Executor

Detailed Purpose & Use:
Executes vector similarity search against the active `VectorStoreManager`. Supports document-specific scoping via `source_filter`. If a specific document is selected, applies vector DB metadata filtering (`{'source': filename}`). Includes fallback oversampling logic (retrieves 3x candidates and manually filters in Python) if native metadata filtering fails or is unsupported by the backend.

Key Exports & Signatures:
```
class Retriever(vector_store: VectorStoreManager, top_k: int = 5) .retrieve_with_scores(query: str,
source_filter: Optional[str] = None) -> List[tuple[Document, float]]
```
Dependencies & Interactions: `VectorStoreManager`, `Document`, `typing`.

Failure Modes & Defenses: Falls back gracefully to post-filtering oversampled results if backend filter kwargs throw exceptions.

File: src/retrieval/retrieval_pipeline.py [CRITICAL]

Role & Layer: Retrieval Workflow Coordinator & Latency Instrumentor

Detailed Purpose & Use:
Coordinates the end-to-end retrieval phase: sanitizes query via `process_query()`, fetches candidates via `Retriever`, normalizes scores and applies threshold via `filter_by_threshold()`, and instruments high-resolution execution timers (`search_time`, `filter_time`, `total_time`). Returns structured payload.

Key Exports & Signatures:
```
retrieve_context(query, vector_store, top_k=5, score_threshold=None, source_filter=None) -> Dict Payload:
{'query': str, 'found': bool, 'results': list, 'message': str, 'metrics': dict}
```
Dependencies & Interactions: `query_processor`, `retriever`, `score_filter`, `src.utils.logger`, `time`.

Failure Modes & Defenses: Catches invalid query ValueErrors and returns a structured failure dict with found=False without crashing.

Chapter 7: Generation & LLM Orchestration (src/generation/)

File: src/generation/__init__.py [LOW]

Role & Layer: Generation Package Marker

Detailed Purpose & Use:
Package marker file for LLM generation components.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/generation/llm_manager.py [CRITICAL]

Role & Layer: Multi-Provider LLM Factory (Groq, OpenAI, Ollama)

Detailed Purpose & Use:
Initializes and manages connections to LLM providers using LangChain interfaces. Supports zero-latency cloud inference via Groq API (`ChatOpenAI` pointing to `https://api.groq.com/openai/v1` with model mapping), commercial cloud inference via OpenAI (`ChatOpenAI` with `OPENAI_API_KEY`), and 100% private local offline inference via Ollama (`ChatOllama` at configurable `OLLAMA_BASE_URL`). Configures model temperatures.

Key Exports & Signatures:
```
class LLMManager(provider='ollama', model_name='llama3', temperature=0.0) .get_llm() -> ChatOllama | ChatOpenAI
```
Dependencies & Interactions: `langchain_ollama.ChatOllama`, `langchain_openai.ChatOpenAI`, `src.utils.logger`, `dotenv`.

Failure Modes & Defenses: Raises `ValueError` if required API keys (`GROQ_API_KEY`, `OPENAI_API_KEY`) are missing from environment.

File: src/generation/prompt_templates.py [CRITICAL]

Role & Layer: LCEL Prompt Templates & Context Formatters

Detailed Purpose & Use:
Constructs LangChain Expression Language (`ChatPromptTemplate`) prompts enforcing strict academic grounding. Defines `RAG_SYSTEM_PROMPT` instructing the LLM to answer using ONLY retrieved context and output explicit refusals if missing. Formats multi-chunk citations with filenames and page numbers. Houses specialized prompts for JSON Multiple Choice Quizzes, JSON Flashcard Decks, and Chapter Summaries.

Key Exports & Signatures:
```
get_rag_prompt_template() -> ChatPromptTemplate format_context_for_prompt(results: list) -> str
get_quiz_prompt_template() -> ChatPromptTemplate get_flashcard_prompt_template() -> ChatPromptTemplate
get_summary_prompt_template() -> ChatPromptTemplate
```
Dependencies & Interactions: `langchain_core.prompts.ChatPromptTemplate`.

Failure Modes & Defenses: None. Stateless string template generators.

File: src/generation/study_generator.py [HIGH]

Role & Layer: Study Tool Synthesis Engine & JSON Parsers

Detailed Purpose & Use:
Generates interactive study materials grounded in document context: MCQs (`generate_quiz`), Flashcards (`generate_flashcards`), and Chapter Summaries (`generate_summary`). Implements `_clean_json_output()` to strip markdown triple backtick fences (```json ... ```) and robustly parses LLM responses into structured Python dictionaries and lists.

Key Exports & Signatures:
```
generate_quiz(topic, vector_store, llm_manager, num_questions=5, source_filter) -> List[Dict]
generate_flashcards(topic, vector_store, llm_manager, num_cards=5, source_filter) -> List[Dict]
generate_summary(topic, vector_store, llm_manager, source_filter) -> str _clean_json_output(text: str) -> str
```
Dependencies & Interactions: `json`, `re`, `llm_manager`, `prompt_templates`, `retrieval_pipeline`, `src.utils.logger`.

Failure Modes & Defenses: Raises `ValueError` if retrieval finds insufficient context (< 0.1 score) or if the LLM fails to return syntactically valid JSON matching the expected schema.

Chapter 8: End-to-End RAG Pipeline (src/rag/)

File: src/rag/__init__.py [LOW]

Role & Layer: RAG Package Marker

Detailed Purpose & Use:
Package marker file for the RAG pipeline.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/rag/rag_pipeline.py [CRITICAL]

Role & Layer: RAG Pipeline Orchestrator & Fallback Handler

Detailed Purpose & Use:
The central orchestrator connecting retrieval and generation. Calls `retrieve_context()`, checks `found` status, and if no relevant chunks exceed the threshold, immediately halts and returns the official anti-hallucination refusal: 'I could not find enough information in the uploaded documents to answer this question.' If context is found, formats citations, invokes the LCEL chain (`prompt | llm | StrOutputParser()`), computes generation and total RAG latency, and returns answer and sources.

Key Exports & Signatures:
```
generate_rag_response(query, vector_store, llm_manager, top_k=5, score_threshold=None, source_filter=None) ->
Dict Payload: {'answer': str, 'sources': list, 'metrics': {'generation_time': float, 'total_rag_time': float,
...}}
```
Dependencies & Interactions: `langchain_core.output_parsers.StrOutputParser`, `retrieval_pipeline`, `prompt_templates`, `time`, `logger`.

Failure Modes & Defenses: Catches LLM generation exceptions and returns an informative user error message without crashing the chat session.

Chapter 9: System Utilities Layer (src/utils/)

File: src/utils/__init__.py [LOW]

Role & Layer: Utilities Package Marker

Detailed Purpose & Use:
Package marker file for utility modules.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/utils/file_utils.py [HIGH]

Role & Layer: SHA-256 File Hasher & Deduplication Engine

Detailed Purpose & Use:
Computes cryptographic SHA-256 checksums of files and raw bytes. Enables document deduplication in `app.py`: if a student re-uploads an identical document, StudyRAG detects the matching hash and reuses existing vector chunks instead of re-processing and duplicating embeddings in the vectorstore.

Key Exports & Signatures:
```
calculate_file_hash(file_content: Union[bytes, Path, str]) -> str
```
Dependencies & Interactions: `hashlib`, `pathlib.Path`, `typing`.

Failure Modes & Defenses: Raises `FileNotFoundError` if path does not exist; raises `ValueError` if input is not bytes, str, or Path.

File: src/utils/history_manager.py [HIGH]

Role & Layer: Study Session Persistence & History CRUD

Detailed Purpose & Use:
Manages persistent logging of user study activity in `data/history.json`. Implements CRUD operations: `load_history()`, `save_history()`, `add_history_entry()`, `delete_history_entry()`, and `clear_all_history()`. Generates unique UUIDv4 identifiers, timestamps, and tracks activity types: 'quiz', 'flashcard', 'summary', 'chat', and 'material'.

Key Exports & Signatures:
```
load_history() -> List[Dict] save_history(items: list) -> bool add_history_entry(item_type, title, details, source) -> Dict delete_history_entry(entry_id: str) -> List[Dict] clear_all_history() -> list
```
Dependencies & Interactions: `json`, `uuid`, `datetime`, `pathlib.Path`, `src.utils.logger`.

Failure Modes & Defenses: Safely handles missing or corrupted JSON files by logging errors and returning empty lists.

File: src/utils/logger.py [MEDIUM]

Role & Layer: Structured Application Logging Subsystem

Detailed Purpose & Use:
Configures application-wide logging with standardized timestamps, logger names, severity levels, and stdout formatting. Prevents duplicate console handlers during Streamlit reactive hot-reloads.

Key Exports & Signatures:
```
setup_logger(name: str = 'studyrag') -> logging.Logger Global instance: logger
```
Dependencies & Interactions: `logging`, `sys`, `pathlib.Path`.

Failure Modes & Defenses: None. Idempotent handler check (`logger.hasHandlers()`) prevents log duplication.

Chapter 10: Evaluation & Benchmarking (src/evaluation/)

File: src/evaluation/__init__.py [LOW]

Role & Layer: Evaluation Package Marker

Detailed Purpose & Use:
Package marker file for the evaluation module.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/evaluation/retrieval_metrics.py [MEDIUM]

Role & Layer: Information Retrieval Benchmark Formulas

Detailed Purpose & Use:
Implements standard Information Retrieval (IR) benchmark metrics to evaluate vector retrieval quality: Precision@K (fraction of top-K retrieved sources matching ground truth: relevant/K), Recall@K (fraction of expected sources found: min(1.0, relevant_count)), and Hit Rate@K (binary indicator: 1 if expected source in top-K, else 0).

Key Exports & Signatures:
```
calculate_precision_at_k(retrieved_sources: List[str], expected_source: str, k: int) -> float
calculate_recall_at_k(retrieved_sources: List[str], expected_source: str, k: int) -> float
calculate_hit_rate_at_k(retrieved_sources: List[str], expected_source: str, k: int) -> int
```
Dependencies & Interactions: `typing`.

Failure Modes & Defenses: Guards against division-by-zero: returns 0.0 if k <= 0.

Chapter 11: User Interface & Component Architecture (src/ui/ & static/)

StudyRAG features a custom, componentized presentation layer that overrides default Streamlit styling. It consists of a dynamic design-token engine, an externalized text dictionary, 12 modular component packages, and global CSS stylesheets.

File: src/ui/__init__.py [LOW]

Role & Layer: UI Package Marker

Detailed Purpose & Use:
Package marker file for the user interface layer.

Key Exports & Signatures:
```
Empty package initializer
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: src/ui/text_provider.py [HIGH]

Role & Layer: Cached UI Text Provider & Localization Engine

Detailed Purpose & Use:
Loads and caches strings from `config/ui_text.yaml` using `@lru_cache(maxsize=1)`. Provides `get_ui_text(*keys, default='')` for nested key access (e.g. `get_ui_text('chat', 'hero_title')`). Ensures instant, zero-IO string retrieval across every Streamlit reactive redraw cycle.

Key Exports & Signatures:
```
load_all_ui_text() -> dict (LRU cached) get_ui_text(*keys, default='') -> Any
```
Dependencies & Interactions: `yaml`, `pathlib.Path`, `functools.lru_cache`.

Failure Modes & Defenses: Returns default string if configuration file is missing or nested key does not exist.

File: src/ui/styles.py [CRITICAL]

Role & Layer: Dynamic CSS Token Engine & Theme Injector

Detailed Purpose & Use:
Manages the application's visual design system. Defines complete CSS root variable sets for DARK_TOKENS and LIGHT_TOKENS (backgrounds, glass borders, accents, button gradients, glow shadows). Scans all component directories (`src/ui/components/**/*.css`), strips comments, minifies CSS, and injects Google Fonts (Plus Jakarta Sans, JetBrains Mono) and styles directly into the Streamlit DOM.

Key Exports & Signatures:
```
collect_component_styles(theme_mode: str = 'dark') -> str inject_app_styles(theme_mode: str = 'dark') -> None
```
Dependencies & Interactions: `streamlit`, `re`, `pathlib.Path`.

Failure Modes & Defenses: Falls back to static CSS if component directories are missing or unreadable.

File: src/ui/components/__init__.py [HIGH]

Role & Layer: Component Facade & Namespace Aggregator

Detailed Purpose & Use:
Central export facade for all 12 modular UI components. Exposes clean render functions to `app.py`, including `render_hero_banner`, `render_doc_item`, `render_citation_box`, `render_quiz_card`, `render_score_card`, `render_flashcard_box`, `render_empty_state`, and `render_welcome_hero`.

Key Exports & Signatures:
All component render functions exported in `__all__` list
Dependencies & Interactions: Imports from all sub-packages in `src/ui/components/*`.

Failure Modes & Defenses: None.

Modular UI Component Subsystems (src/ui/components/)

Subdirectory	Files Included	Render Functions & Visual Responsibilities
base/	base.py variables.css	emit_html(): Raw HTML emitter without wrapper divs. render_spacer(height): Responsive vertical spacing. variables.css: Core layout CSS tokens.
badge/	badge.py badge.css	render_system_pill(): Status indicator badges (Online, Models). render_prompt_chips(): Clickable prompt starter suggestions. render_quiz_badge(): Score and difficulty badges.
button/	button.py button.css	action_button(): Gradient CTA buttons with hover lift animations and icon support.
card/	card.py card.css	render_doc_item(): PDF document card with hash and page count. render_citation_box(): Collapsible citation viewer. render_quiz_card(): Interactive MCQ card with option states. render_flashcard_box(): 3D flip card viewer. render_glass_card_open/close(): Glassmorphic container wrapper.
dashboard/	dashboard.py dashboard.css	render_brand_header(): App branding & active model pill. render_hero_banner(): Page top titles with gradients. render_stat_grid(): Metrics row (Docs, Chunks, Tests). render_footer(): Bottom copyright and links.
emptyState/	empty_state.py empty_state.css	render_empty_state(): Friendly empty state placeholder with SVG icons when no documents or history exist.
errorState/	error_state.py error_state.css	render_error_state(): Polished warning and error alert boxes with action buttons.
input/	input.py input.css	text_input(): Custom styled text inputs and search bars with glowing focus borders.
loading/	loading.py loading.css	render_loading_state(): Shimmer skeletons and gradient spinner animations for async jobs.
notifications/	notifications.py notifications.css	render_notification(): Floating toast alerts for success, warnings, and system info.
tasks/	tasks.py tasks.css	render_task_item(): Background worker task progress item with spin indicators.
welcome/	welcome.py welcome.css	render_welcome_hero(): Large landing banner. render_feature_card_html(): Feature capability cards. render_steps_grid(): 4-step workflow guide.

File: static/css/style.css [HIGH]

Role & Layer: Global Application Stylesheet

Detailed Purpose & Use:
Primary global CSS file containing overarching typography rules, custom scrollbar styling, glassmorphism effects (`backdrop-filter: blur(12px)`), card hover elevation transforms, sidebar overrides, and responsive container constraints.

`Key Exports & Signatures:`
`CSS class definitions (.main-hero, .glass-card, .metric-pill, .chip, etc.)`
Dependencies & Interactions: Read by `styles.py` and consolidated into the Streamlit DOM.

Failure Modes & Defenses: None.

Chapter 12: Automated Test Suite (tests/)

StudyRAG features an extensive, production-grade automated test suite implemented in Pytest. It achieves high code coverage across all core algorithmic modules without requiring external network or live LLM API calls during CI runs.

Test File	Tests	Target Module	Verification Focus & Assertions Tested
test_pdf_loader.py	2	`pdf_loader.py`	Validates `FileNotFoundError` on non-existent files and `ValueError` on invalid extensions (e.g. .txt).
test_document_cleaner.py	4	`document_cleaner.py`	Tests extra space collapsing, newline compression (\n{3,} -> \n\n), empty string handling, and blank page stripping.
test_chunking.py	3	`chunker.py`	Tests recursive splitting, overlap continuity, metadata retention, chunk_id sequencing, and invalid parameter bounds.
test_embeddings.py	2	`embedding_manager.py`	Verifies HuggingFace model initialization, 384-d dimension caching, and cosine similarity ranking of related vs unrelated texts.

test_vectorstore.py	3	`vector_store_manager.py`
		`faiss_store.py`
		`chroma_store.py`
			Tests FAISS and Chroma document additions, index saving to disk, index loading, search result accuracy, and chunk-to-doc mapping.

test_query_processor.py	3	`query_processor.py`	Tests valid natural language query cleaning, whitespace trimming, empty string rejection, and 1,000-char length bound exceptions.
test_retrieval.py	3	`score_filter.py` `retrieval_metrics.py`	Tests FAISS L2 and Chroma Cosine score normalization math, threshold filtering (τ=0.20), Precision@K, Recall@K, and HitRate@K.
test_llm_manager.py	6	`llm_manager.py`	Tests unsupported provider exception handling, Ollama initializations, OpenAI API key validation, and Groq model aliases.
test_rag.py	2	`rag_pipeline.py`	Validates anti-hallucination refusal response when no context meets threshold, and verifies citation formatting when context exists.

Chapter 13: Data & Documentation Artifacts

File: data/history.json [HIGH]

Role & Layer: Persistent Study Activity Datastore

Detailed Purpose & Use:
JSON database recording all student activity across study sessions. Stores an array of activity objects with keys: `id` (UUIDv4), `type` ('quiz', 'flashcard', 'summary', 'chat', 'material'), `title`, `source` (document filter), `timestamp` (human-readable string), and `details` (structured quiz questions, flashcard pairs, summary texts, or chat message logs).

Key Exports & Signatures:
```
JSON Array of study activity records
```
Dependencies & Interactions: Read and updated by `src/utils/history_manager.py`.

Failure Modes & Defenses: If missing, `history_manager.py` initializes an empty array and auto-creates the directory.

File: data/uploaded/ & data/vectorstores/ [HIGH]

Role & Layer: Document Binary Storage & Serialized Vector Indices

Detailed Purpose & Use:
`data/uploaded/` holds uploaded course PDFs (e.g. `dbms.pdf`, `os.pdf`). `data/vectorstores/faiss/` stores serialized FAISS binary indices (`index.faiss` and `index.pkl`). `data/vectorstores/chroma/` stores ChromaDB persistent SQLite databases (`chroma.sqlite3`) and segment folders. Enables persistence across application restarts without re-ingesting documents.

Key Exports & Signatures:
```
Binary vector index files and PDF source files
```
Dependencies & Interactions: Managed by `VectorStoreManager` and `app.py`.

Failure Modes & Defenses: If corrupted or deleted, `app.py` automatically re-indexes uploaded PDFs on startup.

File: docs/PROJECT_DOCUMENTATION.tex & docs/temp_sanitized.md [MEDIUM]

Role & Layer: Master Reference Documentation & LaTeX Specifications

Detailed Purpose & Use:
`docs/temp_sanitized.md` is an exhaustive 23-chapter master technical handover document covering theoretical foundations, cognitive science motivation (testing effect, spaced retrieval), mathematical algorithms, and module implementations. `docs/PROJECT_DOCUMENTATION.tex` is its publication-grade LaTeX compilation.

Key Exports & Signatures:
```
Markdown and LaTeX reference documentation
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

File: documentation/PROJECT_CARD.docx [LOW]

Role & Layer: Executive Project Summary Card

Detailed Purpose & Use:
Executive Microsoft Word (.docx) document summarizing project goals, architecture, feature set, pedagogical objectives, and technical stack for institutional evaluation and portfolio presentation.

Key Exports & Signatures:
```
Word document artifact
```
Dependencies & Interactions: None.

Failure Modes & Defenses: None.

Chapter 14: Master File Quick-Reference Matrix

The following master lookup table summarizes every file in StudyRAG, its architectural layer, its primary functional responsibility, and its operational criticality:

File Path	Layer	Primary Responsibility	Criticality
app.py	Presentation	Streamlit entry point, UI routing, worker threads	**CRITICAL**
config/settings.yaml	Config	Default chunking and retrieval parameters	**CRITICAL**
config/ui_text.yaml	Config	Externalized UI text, labels, and tooltips	**HIGH**
requirements.txt	Build	Python package dependency manifest	**CRITICAL**
pytest.ini	Test	Pytest configuration and pythonpath setting	**MEDIUM**
src/ingestion/pdf_loader.py	Ingestion	PyMuPDF page extraction and validation	**CRITICAL**
src/ingestion/document_cleaner.py	Ingestion	Whitespace normalization and blank page removal	**CRITICAL**
src/ingestion/chunker.py	Ingestion	Recursive character splitting with metadata	**CRITICAL**
src/ingestion/ingestion_pipeline.py	Ingestion	End-to-end PDF processing coordinator	**CRITICAL**
src/embeddings/embedding_manager.py	Embeddings	Local HuggingFace model manager & caching	**CRITICAL**
src/vectorstore/vector_store_manager.py	Vector Store	Unified factory interface for FAISS and Chroma	**CRITICAL**
src/vectorstore/faiss_store.py	Vector Store	In-memory L2 search and disk serialization	**HIGH**
src/vectorstore/chroma_store.py	Vector Store	Persistent SQLite and HNSW cosine store	**HIGH**
src/retrieval/query_processor.py	Retrieval	Query validation, sanitization, length bounds	**CRITICAL**
src/retrieval/score_filter.py	Retrieval	Score normalization and anti-hallucination filter	**CRITICAL**
src/retrieval/retriever.py	Retrieval	Source-filtered nearest-neighbor search	**CRITICAL**
src/retrieval/retrieval_pipeline.py	Retrieval	Retrieval workflow coordinator with latency metrics	**CRITICAL**
src/generation/llm_manager.py	Generation	Groq, OpenAI, and Ollama multi-model factory	**CRITICAL**
src/generation/prompt_templates.py	Generation	LCEL prompt templates for RAG, Quizzes, Cards	**CRITICAL**
src/generation/study_generator.py	Generation	MCQ, Flashcard, and Summary generation engine	**HIGH**
src/rag/rag_pipeline.py	RAG Core	End-to-end RAG orchestrator with fallback refusal	**CRITICAL**
src/utils/file_utils.py	Utils	SHA-256 chunked hashing & deduplication	**HIGH**
src/utils/history_manager.py	Utils	JSON history CRUD operations and UUID tracking	**HIGH**
src/utils/logger.py	Utils	Centralized Python logger configuration	**MEDIUM**
src/evaluation/retrieval_metrics.py	Evaluation	Precision@K, Recall@K, HitRate benchmarks	**MEDIUM**
src/ui/text_provider.py	UI Engine	LRU-cached UI text provider	**HIGH**
src/ui/styles.py	UI Engine	Dynamic CSS token engine & DOM style injector	**CRITICAL**

src/ui/components/*.py	UI Components	12 modular UI packages (cards, pills, buttons, etc.)	**HIGH**
static/css/style.css	Static Web	Global base styles, glassmorphism, scrollbars	**HIGH**
tests/test_*.py (9 modules)	Test Suite	Automated unit and integration test suite	**HIGH**
data/history.json	Data Store	Persistent JSON study session logs	**HIGH**
data/vectorstores/*	Data Store	Serialized FAISS indices and Chroma SQLite DB	**HIGH**

HANDOVER VERIFICATION COMPLETE

Document Sign-off: This technical specification handbook represents the complete, verified state of the StudyRAG repository. Every file has been cross-referenced against production code, test suites, and runtime logs. For further modifications, refer to Chapter 2 for configuration hyperparameters and Chapter 12 for automated test verification commands (pytest).